

NFS con no_root_squash

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

no_root_squash es una opción de NFS que hace que el root de tu máquina sea tratado como root en el servidor NFS (en lugar de degradarlo a nobody). Si un share exportado con esa opción está montado en una ruta accesible en la víctima, montas el share desde tu Kali (donde SI eres root), creas ahí un binario SUID root, y lo ejecutas en la víctima para escalar.

Normalmente NFS 'aplasta' (squash) el root remoto para que no pueda actuar como root local. no_root_squash desactiva esa protección: los ficheros que creas como root en tu máquina conservan uid 0 en el servidor. Un binario SUID creado así corre como root al ejecutarlo en la víctima.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ves NFS (2049) o /etc/exports revela un share con no_root_squash. showmount lista los exports disponibles.

Cómo identificarla

- showmount -e muestra un export disponible.
- /etc/exports (legible en la víctima) contiene 'no_root_squash'.
- El export está montado o es montable en una ruta útil de la víctima.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Monta el share en tu Kali, crea un binario SUID como root, y ejecútalo desde la víctima por esa misma ruta.

Evidencia esperada: El binario mantiene el bit SUID y euid root al ejecutarse en la víctima.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
showmount -e $IP
```

Objetivo y contexto: Lista los directorios que el servidor NFS exporta y a quien. Es el primer paso: necesitas saber que share montar y si aplica.

Resultado esperado: Rutas exportadas y sus restricciones de host. Un export sin restriccion (*) es montable por ti. Luego confirma no_root_squash en /etc/exports.

Capa 2: prueba documentada

```
sudo mount -t nfs $IP:/export /mnt/nfs && cd /mnt/nfs
```

Objetivo y contexto: Monta el share en tu Kali, donde eres root. Los ficheros SUID que crees aqui como root conservaran uid 0 en la victima gracias a no_root_squash.

Resultado esperado: El share montado en /mnt/nfs. Ahora compila un binario que lance bash, hazlo chown root:root y chmod +s; luego ejecutalo en la victima desde esa misma ruta.

Anatomía de los payloads

La primera prueba es showmount -e \$IP. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El binario mantiene el bit SUID y euid root al ejecutarse en la victima.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera los exports con showmount -e contra la IP.
- Monta el share en tu Kali (como root).
- Crea ahí un binario que lance una shell y ponle el bit SUID (como root de tu Kali).
- En la victima, ejecuta ese binario desde la ruta del share: correra como root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- showmount -e no devuelve ningun export, o el export tiene root_squash activo.
- No puedes montar NFS desde tu maquina de ataque (firewall, sin cliente NFS).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un export NFS con no_root_squash te deja crear ficheros SUID como root desde tu maquina y ejecutarlos en la victima. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (nfs-no-root-squash)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.